

Sistemas Distribuídos e Computação em Nuvem

Aula 17 — Revisão da C3 + Avaliação C3.A1

C3 ·  **DIA DE AVALIAÇÃO** · entrega do C3.A2 (projeto integrador)

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Como funciona o dia de hoje

1. **Revisão consolidada** da C3 (Aulas 13–16) e **o semestre inteiro** em uma página.
2. **Esquenta** — 5 questões comentadas no estilo ENADE.
3. **Avaliação C3.A1** — prova escrita, estilo ENADE (**5,0 pontos**).
4. **Entrega do C3.A2** — Plataforma de IA como Serviço no repositório (**5,0 pontos**).

 **Nota da C3 = C3.A1 (5,0) + C3.A2 (5,0).** Hoje fecham as duas — e o semestre.

O caminho que percorremos na C3

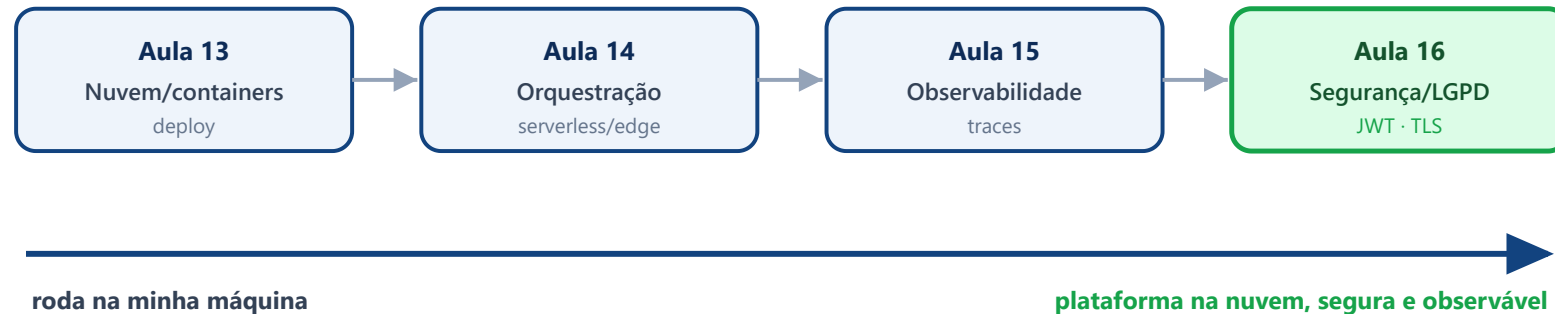
A pergunta da C3: **como levar o sistema para a nuvem, de forma escalável, observável e segura?**

- **Aula 13 — nuvem e containers:** IaaS/PaaS/SaaS, imagem/container, 1º deploy.
- **Aula 14 — orquestração e serverless:** docker-compose, FaaS/cold start, borda.
- **Aula 15 — observabilidade:** logs, métricas e **traces** (OpenTelemetry).
- **Aula 16 — segurança e LGPD:** TLS/HTTPS, **JWT**, minimização de dados.

 Releia os quadros de cada aula e confira o seu C3.A2 contra a rubrica.

A trilha da C3 — da sua máquina à nuvem segura

a pergunta da C3: como levar o sistema à nuvem de forma escalável, observável e segura?



💡 A C3 transformou o sistema da C2 numa **plataforma de verdade**: empacotada, escalável, monitorada e protegida.

O semestre inteiro em uma página

C1 · Comunicação (1–7)

Falha parcial · socket · TCP/UDP · thread/corrída · gRPC/.proto · REST/status/idempotência · IA como serviço.

C2 · Coordenação (8–12)

Fila/assíncrono · Lamport/vetorial · CAP (CP×AP) · Raft/quórum · resiliência (timeout/retry/disjuntor).

C3 · Nuvem e segurança (13–18)

IaaS/PaaS/SaaS · container/imagem · elasticidade · orquestração · serverless/cold start · observabilidade (log/métrica/trace) · TLS/JWT/LGPD.

💡 Um fio só: um **serviço de IA** que cresceu de um socket a uma **plataforma cloud-native**.

As 6 perguntas que você precisa saber responder (C3)

1. Qual a diferença entre **IaaS**, **PaaS** e **SaaS** (quem gerencia o quê)?
2. O que um **container** compartilha que uma **VM** não — e por que é mais leve?
3. O que é **serverless** e qual o seu **preço** (cold start, sem estado)?
4. Quais os **três pilares** da observabilidade e o que o **trace** responde?
5. **Autenticação** × **autorização**: o que o **JWT** garante — e o que **não** garante?
6. O que a **LGPD** exige no pipeline de IA (**minimização**)?

 Travou em alguma? Volte ao deck da aula correspondente **agora**.

Esquenta C3

5 questões comentadas — estilo ENADE

Questão 1 — modelo de serviço

Uma startup quer publicar sua aplicação **enviando apenas o código**, sem gerenciar sistema operacional nem servidores, e com **escala automática**. O modelo de nuvem mais adequado é:

A) IaaS · B) **PaaS** · C) SaaS · D) on-premises · E) nenhum

Questão 1 — resposta B

- **PaaS**: a plataforma cuida de S.O., runtime e escala; você entrega **só o código** (Render, Railway, App Engine).
- IaaS entregaria a VM crua (você gerencia o S.O.); SaaS é software pronto ao usuário final.

💡 Aula 13 — "enviar só o código" é a assinatura do PaaS.

Questão 2 — container × VM

Sobre a diferença entre **container** e **máquina virtual**, é correto afirmar que o container:

- A) carrega um sistema operacional convidado completo, como a VM
- B) **compartilha o kernel/S.O. do host, sendo mais leve e rápido de subir**
- C) só roda em sistemas Windows
- D) elimina a necessidade de imagem
- E) é sempre mais seguro que qualquer VM

Questão 2 — resposta B

- O container **compartilha o S.O. do host** → sobe em segundos e pesa MB; a VM carrega um **S.O. convidado inteiro** (minutos, GB).
- Precisa de **imagem** (D é falso) e roda em qualquer host com motor de containers (C é falso).

💡 Aula 13 — container × VM. É a pegadinha mais comum do bloco.

Questão 3 — serverless

Uma função **serverless** demora mais **na primeira** invocação após um período ocioso; as seguintes são rápidas. Esse atraso é o:

A) timeout · B) deadlock · C) **cold start** · D) circuit breaker · E) clock drift

Questão 3 — resposta C

- **Cold start:** a plataforma precisa **inicializar o ambiente** da função que havia escalado a zero. A 1ª paga esse custo; as seguintes reaproveitam a instância quente.

💡 Aula 14 (e Aula 6) — serverless troca servidor ocioso por cold start ocasional.

Questão 4 — observabilidade

Em uma requisição que passa por **quatro serviços**, a equipe precisa saber **em qual deles** o tempo é gasto, **requisição a requisição**. O recurso adequado é:

- A) só logs lidos separadamente
- B) só métricas de CPU
- C) **rastreamento distribuído (traces) com `trace_id` propagado**
- D) aumentar o log para DEBUG
- E) reiniciar os serviços

Questão 4 — resposta C

- Só o **trace** liga as etapas de **uma mesma** requisição (via `trace_id`) e mostra **onde** o tempo foi gasto. Logs soltos não costuram a jornada; métricas dão o agregado.

💡 Aula 15 — para "onde está lento nesta requisição?", a resposta é tracing.

Questão 5 — segurança (JWT)

Um desenvolvedor guarda a **senha** do usuário no **payload do JWT**, achando que "está protegida". Sobre isso:

- A) correto, o JWT é cifrado
- B) **errado: o payload é apenas codificado (base64) e legível; a assinatura garante integridade, não sigilo**
- C) correto, se o token expirar
- D) errado, pois JWT não aceita campos extras
- E) correto, pois o HTTPS protege tudo

Questão 5 — resposta B

- O **payload** do JWT é **base64** (legível por qualquer um). A **assinatura** garante que não foi **adulterado**, não que seja **secreto**.
- Logo, **nunca** coloque senha/segredo no payload.

💡 Aula 16 — integridade $\neq$ sigilo. É o que o `auth_demo.py` mostra.

Avaliação C3.A1 — a prova

- **Escrita, individual, estilo ENADE** — vale **5,0 pontos**.
- **Integra a C3** (nuvem, containers, orquestração, serverless, observabilidade, segurança) e **puxa** conceitos da C1/C2 em estudos de caso.
- **Cobra entendimento operacional**: classificar o modelo, escolher o recurso certo, apontar o trade-off.

 As 5 questões do esquentão são do mesmo estilo da prova. Se você as entendeu, está pronto.

Entrega do C3.A2 — checklist antes de submeter

Dia de entrega do **projeto integrador**. Rode do zero e confira contra a **rubrica**:

- **Microserviços** (gateway + inferência + worker + vetorial) subindo com **um comando**?
- **Deploy** na nuvem com **URL pública** e **HTTPS**?
- **Resiliência** (circuit breaker/retry/idempotência) e **observabilidade** (trace)?
- **Segurança** (JWT) e **LGPD** (minimização) no README? **Relatório de trade-offs** (CAP, custo, latência)?

💡 Rubrica: arquitetura **1,5** · comunicação **1,5** · resiliência **1,0** · execução reproduzível **1,0**. A sofisticação do modelo de IA não pontua.

◆ Foco ENADE

- A **C3.A1** integra o bloco e **amarra** o semestre.
- No ENADE, **nuvem, containers, serverless e segurança** aparecem como estudos de caso — e **misturados** com comunicação (C1) e coordenação (C2).
- **Treine**: classificar IaaS/PaaS/SaaS, container×VM, escolher o recurso de observabilidade, apontar o que o JWT (não) garante.

Termos-chave: IaaS/PaaS/SaaS · Container · Serverless/cold start · Observabilidade/trace · JWT · TLS · LGPD

💡 O ENADE é **neste domingo (29/11)**. Releia os quadros *Foco ENADE* de todas as 17 aulas — é o resumo definitivo.

Fora da sala · Glossário

Para revisar

- Os quadros de resumo das Aulas 13–16 (e todos os *Foco ENADE*).
- Os exemplos: `app.py` / `Dockerfile`, `docker-compose`, `tracing_demo.py`, `auth_demo.py`.
- O seu **C3.A2** contra a **rubrica**.

Glossário

- **C3.A1**: avaliação escrita individual, estilo ENADE (5,0).
- **C3.A2**: Plataforma de IA como Serviço no repositório (5,0).
- **Cloud-native**: sistema desenhado para a nuvem (containers, escala, resiliência).
- **Trade-off**: troca consciente (ex.: consistência × disponibilidade).

Boa prova! 🚀

Entregue o C3.A2 e arrase no ENADE (29/11).

Próxima (Aula 18): Fechamento do semestre — o que você construiu, para onde a área está indo e os próximos passos.

← Anterior

Aula 16 · Segurança/LGPD

≡ Índice

todas as aulas